In the name of Allah

Data Networks

Prof. Mohammad Reza Pakravan

Assignment 3

Farzan Aghaei 401101202

Sharif University of Technology

Electrical Engineering Department

1 Ethernet

1.1 Switch and Hub

a)

Ethernet Hub: A simple multi-port repeater that blindly copies every incoming electrical signal out to every other port.

Ethernet Switch: An intelligent multi-port bridge that reads Ethernet frame headers, learns where each MAC address lives, and forwards traffic only to the correct port(ports).

Because a Hub simply sends every bit it hears ti every other port, it's still used in network labs and also many industrial or classroom setups keep hubs around to use the old 10Mb half-duplex gear or to demonstrate collisions.

A switch learns MAC addresses and forwards each frame only where it belongs, so every port gets its own collision-free, often full-duplex bandwidth plus extras like VLANs and PoE. That makes switches common in all kinds of network such as: office wiring closets, industrial IoT cabinets and even the 8-port plastic box under a smart-TV stand.

b)

First I created the Network Description shown in the figure as below:

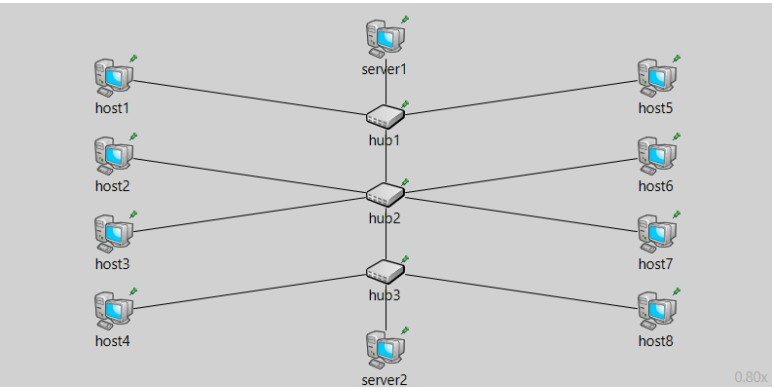


Figure 1: Network Description of 1.1 parts a to c

The next step is to construct the initialization file(.ini). I added the mentioned configurations to each host and server and then established the needed frame transmissions. The code is shown in the following figure.

```
[Q1_Partb]
network = topology.Part1_a
sim-time-limit = 1s

Part1_a.*.cli.sendInterval = exponential(50ms)
Part1_a.*.cli.reqLength = intuniform(800B, 1200B)
Part1_a.*.cli.respLength = intuniform(10B, 30B)

Part1_a.host1.cli.destAddress = "host4"
Part1_a.host3.cli.destAddress = "host2"
Part1_a.host8.cli.destAddress = "host1"
```

Figure 2: Initialization code for the needed configurations

Then I simulated the Network.

The results I got from the simulation are displayed in figures 3 and 12.

Experiment	Measurement	Replication	Module	Name	Value
Q1_Partb		#0	Part1_a.host1.eth.mac	collision:count	2
Q1_Partb		#0	Part1_a.host2.eth.mac	collision:count	3
Q1_Partb		#0	Part1_a.host3.eth.mac	collision:count	0
Q1_Partb		#0	Part1_a.host4.eth.mac	collision:count	3
Q1_Partb		#0	Part1_a.host5.eth.mac	collision:count	3
Q1_Partb		#0	Part1_a.host6.eth.mac	collision:count	3
Q1_Partb		#0	Part1_a.host7.eth.mac	collision:count	3
Q1_Partb		#0	Part1_a.host8.eth.mac	collision:count	3
Q1_Partb		#0	Part1_a.server1.eth.mac	collision:count	2
Q1_Partb		#0	Part1_a.server2.eth.mac	collision:count	2

Figure 3: The collision count in each node

Experiment	Measurement	Replication	Module	Name	Value
Q1_Partb		#0	Part1_a.host1.eth.mac	rx channel utilization (%)	3.854 96
Q1_Partb		#0	Part1_a.host2.eth.mac	rx channel utilization (%)	5.260 8
Q1_Partb		#0	Part1_a.host3.eth.mac	rx channel utilization (%)	3.246 32
Q1_Partb		#0	Part1_a.host4.eth.mac	rx channel utilization (%)	5.157 12
Q1_Partb		#0	Part1_a.host5.eth.mac	rx channel utilization (%)	5.260 8
Q1_Partb		#0	Part1_a.host6.eth.mac	rx channel utilization (%)	5.260 8
Q1_Partb		#0	Part1_a.host7.eth.mac	rx channel utilization (%)	5.260 8
Q1_Partb		#0	Part1_a.host8.eth.mac	rx channel utilization (%)	3.930 24
Q1_Partb		#0	Part1_a.server1.eth.mac	rx channel utilization (%)	5.250 88
Q1_Partb		#0	Part1_a.server2.eth.mac	rx channel utilization (%)	5.197 84

Figure 4: Utilization of each node

As we can see from the results, the total count of collisions and the mean utilization of the nodes were 24(2.4 average) and 4.75% respectively.

Theoretical calculations:

$$\text{request transmission time} : \frac{1000B \times 8}{10 \times 10^6} = 0.8msec$$

$$\text{response transmission time} : \frac{20B \times 8}{10 \times 10^6} = 0.016msec$$

$$\text{request generation rate} : 3 \times \frac{1}{0.05} = 60req/sec$$

$$\text{total transmission bandwidth} : 60 \times (1000 + 20) \times 8 = 0.4896Mbps$$

$$\Rightarrow \text{channel utilization} : \frac{0.4896}{10} \times 100 \approx 4.9\%(\text{collision free})$$

So as I derived the theoretical and simulation results are close to each other and the little difference is because of the randomness of the packet length and also the sending intervals.

c)

I changed the *sendInterval* parameter to *exponential(10ms)* and then repeated the process of the previous part.

The obtained results:

Experiment	Measurement	Replication	Module	Name	Value
Q1_Partc		#0	Part1_a.host1.eth.mac	collision:count	26
Q1_Partc		#0	Part1_a.host2.eth.mac	collision:count	50
Q1_Partc		#0	Part1_a.host3.eth.mac	collision:count	22
Q1_Partc		#0	Part1_a.host4.eth.mac	collision:count	46
Q1_Partc		#0	Part1_a.host5.eth.mac	collision:count	50
Q1_Partc		#0	Part1_a.host6.eth.mac	collision:count	50
Q1_Partc		#0	Part1_a.host7.eth.mac	collision:count	50
Q1_Partc		#0	Part1_a.host8.eth.mac	collision:count	24
Q1_Partc		#0	Part1_a.server1.eth.mac	collision:count	46
Q1_Partc		#0	Part1_a.server2.eth.mac	collision:count	46

Figure 5: The collision count in each node

Experiment	Measurement	Replication	Module	Name	Value
Q1_Partc		#0	Part1_a.host1.eth.mac	rx channel utilization (%)	~ 13.951 5
Q1_Partc		#0	Part1_a.host2.eth.mac	rx channel utilization (%)	~ 16.938 5
Q1_Partc		#0	Part1_a.host3.eth.mac	rx channel utilization (%)	~ 13.533 4
Q1_Partc		#0	Part1_a.host4.eth.mac	rx channel utilization (%)	~ 16.950 2
Q1_Partc		#0	Part1_a.host5.eth.mac	rx channel utilization (%)	~ 16.938 5
Q1_Partc		#0	Part1_a.host6.eth.mac	rx channel utilization (%)	~ 16.938 5
Q1_Partc		#0	Part1_a.host7.eth.mac	rx channel utilization (%)	~ 16.938 5
Q1_Partc		#0	Part1_a.host8.eth.mac	rx channel utilization (%)	13.644
Q1_Partc		#0	Part1_a.server1.eth.mac	rx channel utilization (%)	~ 16.922 1
Q1_Partc		#0	Part1_a.server2.eth.mac	rx channel utilization (%)	~ 16.885 4

Figure 6: Utilization of each node

It is observed that the total collisions went from 24 up to 410 and also the utilization rose to 15.96%.

The increase in the number of collisions is because the *sendInterval* is decreased from 50msec to 10msec. This means the channel is now 5 times busier, and packets are far more likely to overlap in time, especially since each hub simply broadcasts to all ports. As the channel becomes busier, the chance that two nodes will transmit at nearly the same time rises exponentially, causing collision rates to grow remarkably.

Theoretical calculations:

$$\text{request transmission time} : \frac{1000B \times 8}{10 \times 10^6} = 0.8msec$$

$$\text{response transmission time} : \frac{20B \times 8}{10 \times 10^6} = 0.016msec$$

$$\text{request generation rate} : 3 \times \frac{1}{0.01} = 300req/sec$$

$$\text{total transmission bandwidth} : 300 \times (1000 + 20) \times 8 = 2.448Mbps$$

$$\Rightarrow \text{channel utilization} : \frac{2.448}{10} \times 100 \approx 24.48\%(\text{collision free})$$

As obvious in the calculations, the ideal utilization(without any collisions) is 24.48%. But in this case because we have a notable collision rate(unlike with the previous part where the collision rate was negligible) the actual utilization is less than the theoretical value.

d)

Now in this part I changed the Ethernet Hubs into Ethernet Switches and then repeated the previous parts.

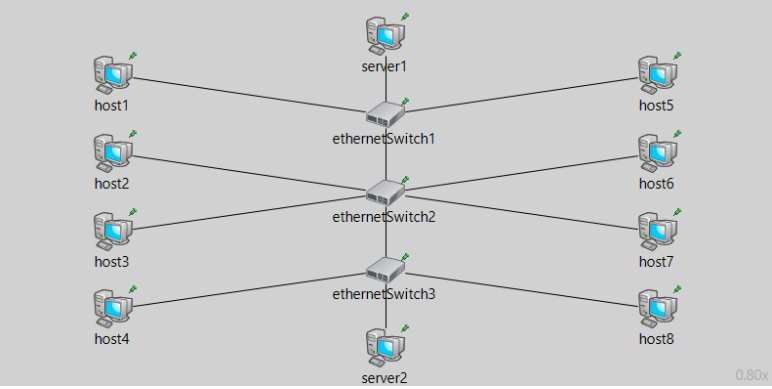


Figure 7: Network description of 1.1 part d

```
24
25 [Q1_Partdb]
26 network = topology.Part1_d
27 sim-time-limit = 1s
28
29 Part1_d.*.cli.sendInterval = exponential(50ms)
30 Part1_d.*.cli.reqLength = intuniform(800B, 1200B)
31 Part1_d.*.cli.respLength = intuniform(10B, 30B)
32
33 Part1_d.host1.cli.destAddress = "host4"
34 Part1_d.host3.cli.destAddress = "server2"
35 Part1_d.host8.cli.destAddress = "server1"
36
```

Figure 8: Initialization code for the needed configurations

sendInterval = exponential(50ms):

Experiment	Measurement	Replication	Module	Name	Value
Q1_Partdb		#0	Part1_d.host1.eth.mac	collision:count	0
Q1_Partdb		#0	Part1_d.host2.eth.mac	collision:count	0
Q1_Partdb		#0	Part1_d.host3.eth.mac	collision:count	0
Q1_Partdb		#0	Part1_d.host4.eth.mac	collision:count	0
Q1_Partdb		#0	Part1_d.host5.eth.mac	collision:count	0
Q1_Partdb		#0	Part1_d.host6.eth.mac	collision:count	0
Q1_Partdb		#0	Part1_d.host7.eth.mac	collision:count	0
Q1_Partdb		#0	Part1_d.host8.eth.mac	collision:count	0
Q1_Partdb		#0	Part1_d.server1.eth.mac	collision:count	0
Q1_Partdb		#0	Part1_d.server2.eth.mac	collision:count	0

Figure 9: The collision count in each node

Experiment	Measurement	Replication	Module	Name	Value
Q1_Partdb		#0	Part1_d.host1.eth.mac	rx channel utilization (%)	0.276
Q1_Partdb		#0	Part1_d.host2.eth.mac	rx channel utilization (%)	0.255 04
Q1_Partdb		#0	Part1_d.host3.eth.mac	rx channel utilization (%)	0.348 8
Q1_Partdb		#0	Part1_d.host4.eth.mac	rx channel utilization (%)	1.730 96
Q1_Partdb		#0	Part1_d.host5.eth.mac	rx channel utilization (%)	0.255 04
Q1_Partdb		#0	Part1_d.host6.eth.mac	rx channel utilization (%)	0.255 04
Q1_Partdb		#0	Part1_d.host7.eth.mac	rx channel utilization (%)	0.255 04
Q1_Partdb		#0	Part1_d.host8.eth.mac	rx channel utilization (%)	0.253 92
Q1_Partdb		#0	Part1_d.server1.eth.mac	rx channel utilization (%)	1.492 32
Q1_Partdb		#0	Part1_d.server2.eth.mac	rx channel utilization (%)	2.529 6
Q1_Partdb		#0	Part1_d.ethernetSwitch1.eth[0].mac	rx channel utilization (%)	1.564 4
Q1_Partdb		#0	Part1_d.ethernetSwitch1.eth[1].mac	rx channel utilization (%)	0
Q1_Partdb		#0	Part1_d.ethernetSwitch1.eth[2].mac	rx channel utilization (%)	0.092 16
Q1_Partdb		#0	Part1_d.ethernetSwitch1.eth[3].mac	rx channel utilization (%)	1.513 28
Q1_Partdb		#0	Part1_d.ethernetSwitch2.eth[0].mac	rx channel utilization (%)	0
Q1_Partdb		#0	Part1_d.ethernetSwitch2.eth[1].mac	rx channel utilization (%)	2.347 84
Q1_Partdb		#0	Part1_d.ethernetSwitch2.eth[2].mac	rx channel utilization (%)	0
Q1_Partdb		#0	Part1_d.ethernetSwitch2.eth[3].mac	rx channel utilization (%)	0
Q1_Partdb		#0	Part1_d.ethernetSwitch2.eth[4].mac	rx channel utilization (%)	1.656 56
Q1_Partdb		#0	Part1_d.ethernetSwitch2.eth[5].mac	rx channel utilization (%)	1.607 04
Q1_Partdb		#0	Part1_d.ethernetSwitch3.eth[0].mac	rx channel utilization (%)	0.109 44
Q1_Partdb		#0	Part1_d.ethernetSwitch3.eth[1].mac	rx channel utilization (%)	1.330 56
Q1_Partdb		#0	Part1_d.ethernetSwitch3.eth[2].mac	rx channel utilization (%)	0.167 04
Q1_Partdb		#0	Part1_d.ethernetSwitch3.eth[3].mac	rx channel utilization (%)	4.004 4

Figure 10: Utilization of each node

sendInterval = exponential(10ms):

Experiment	Measurement	Replication	Module	Name	Value
Q1_Partdc		#0	Part1_d.host1.eth.mac	collision:count	0
Q1_Partdc		#0	Part1_d.host2.eth.mac	collision:count	0
Q1_Partdc		#0	Part1_d.host3.eth.mac	collision:count	0
Q1_Partdc		#0	Part1_d.host4.eth.mac	collision:count	0
Q1_Partdc		#0	Part1_d.host5.eth.mac	collision:count	0
Q1_Partdc		#0	Part1_d.host6.eth.mac	collision:count	0
Q1_Partdc		#0	Part1_d.host7.eth.mac	collision:count	0
Q1_Partdc		#0	Part1_d.host8.eth.mac	collision:count	0
Q1_Partdc		#0	Part1_d.server1.eth.mac	collision:count	0
Q1_Partdc		#0	Part1_d.server2.eth.mac	collision:count	0

Figure 11: The collision count in each node

Experiment	Measurement	Replication	Module	Name	Value
Q1_Partdc		#0	Part1_d.host1.eth.mac	rx channel utilization (%)	0.758 32
Q1_Partdc		#0	Part1_d.host2.eth.mac	rx channel utilization (%)	0.328 4
Q1_Partdc		#0	Part1_d.host3.eth.mac	rx channel utilization (%)	0.780 8
Q1_Partdc		#0	Part1_d.host4.eth.mac	rx channel utilization (%)	7.703 6
Q1_Partdc		#0	Part1_d.host5.eth.mac	rx channel utilization (%)	0.328 4
Q1_Partdc		#0	Part1_d.host6.eth.mac	rx channel utilization (%)	0.328 4
Q1_Partdc		#0	Part1_d.host7.eth.mac	rx channel utilization (%)	0.328 4
Q1_Partdc		#0	Part1_d.host8.eth.mac	rx channel utilization (%)	0.776 56
Q1_Partdc		#0	Part1_d.server1.eth.mac	rx channel utilization (%)	8.281 84
Q1_Partdc		#0	Part1_d.server2.eth.mac	rx channel utilization (%)	8.784 16
Q1_Partdc		#0	Part1_d.ethernetSwitch1.eth[0].mac	rx channel utilization (%)	7.463 68
Q1_Partdc		#0	Part1_d.ethernetSwitch1.eth[1].mac	rx channel utilization (%)	0
Q1_Partdc		#0	Part1_d.ethernetSwitch1.eth[2].mac	rx channel utilization (%)	0.541 44
Q1_Partdc		#0	Part1_d.ethernetSwitch1.eth[3].mac	rx channel utilization (%)	8.711 76
Q1_Partdc		#0	Part1_d.ethernetSwitch2.eth[0].mac	rx channel utilization (%)	0
Q1_Partdc		#0	Part1_d.ethernetSwitch2.eth[1].mac	rx channel utilization (%)	8.602 4
Q1_Partdc		#0	Part1_d.ethernetSwitch2.eth[2].mac	rx channel utilization (%)	0
Q1_Partdc		#0	Part1_d.ethernetSwitch2.eth[3].mac	rx channel utilization (%)	0
Q1_Partdc		#0	Part1_d.ethernetSwitch2.eth[4].mac	rx channel utilization (%)	8.005 12
Q1_Partdc		#0	Part1_d.ethernetSwitch2.eth[5].mac	rx channel utilization (%)	9.164 16
Q1_Partdc		#0	Part1_d.ethernetSwitch3.eth[0].mac	rx channel utilization (%)	0.518 4
Q1_Partdc		#0	Part1_d.ethernetSwitch3.eth[1].mac	rx channel utilization (%)	8.046 72
Q1_Partdc		#0	Part1_d.ethernetSwitch3.eth[2].mac	rx channel utilization (%)	0.599 04
Q1_Partdc		#0	Part1_d.ethernetSwitch3.eth[3].mac	rx channel utilization (%)	16.607 5

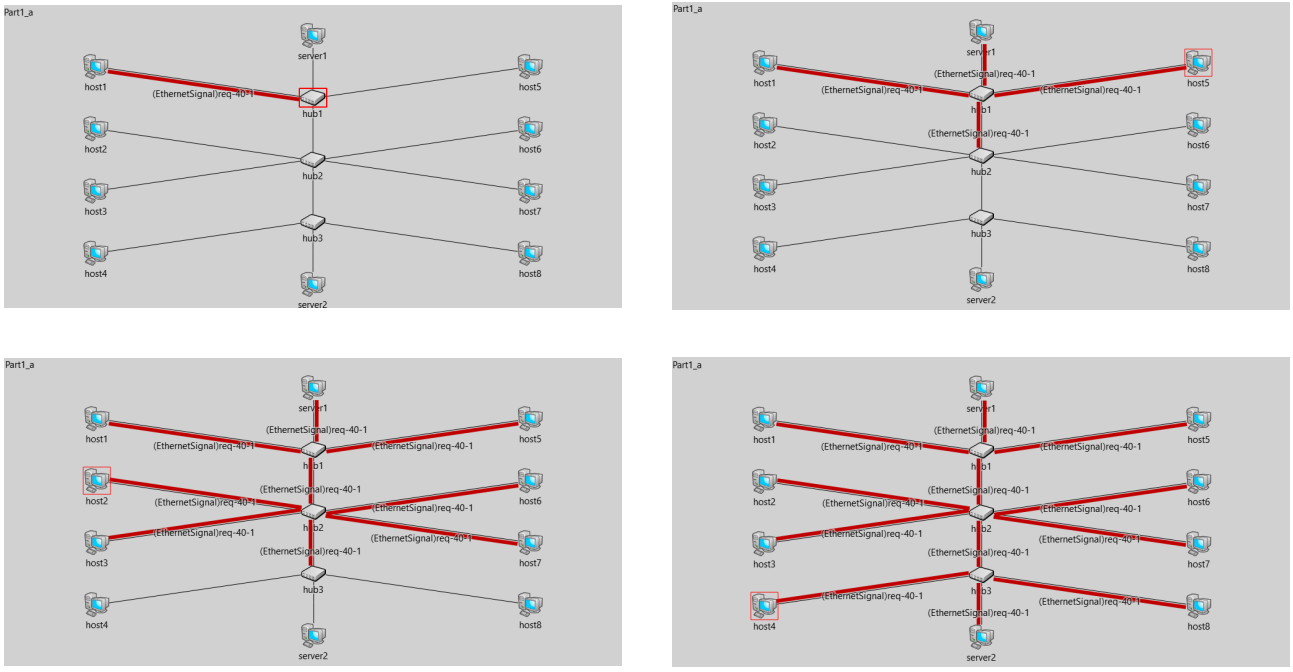
Figure 12: Utilization of each node

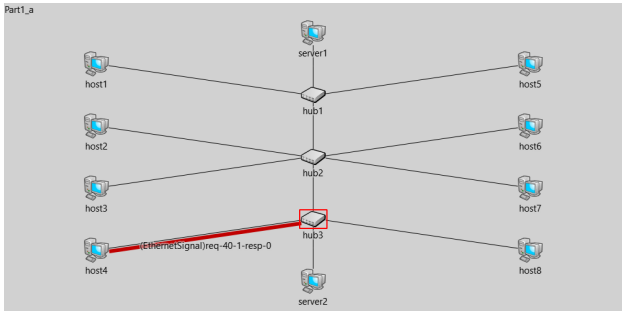
As seen from the results the collision rate is zero! This is because Ethernet Switches create isolated collision domains and each port is full-duplex and devices don't content for a shared medium anymore. Thus regardless of traffic rate the no collision will occur.

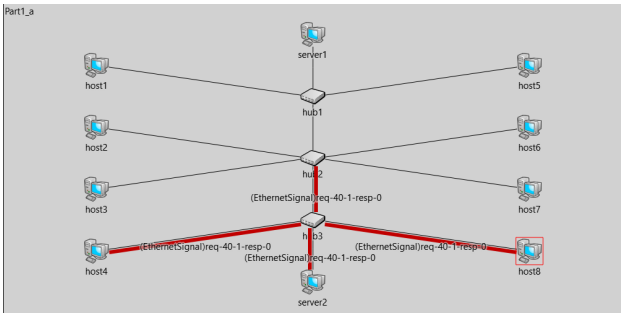
The mean utilization of the nodes increased from nearly 0.9% to around 4.1% which is due to the traffic rate being 5 times higher in the latter case.

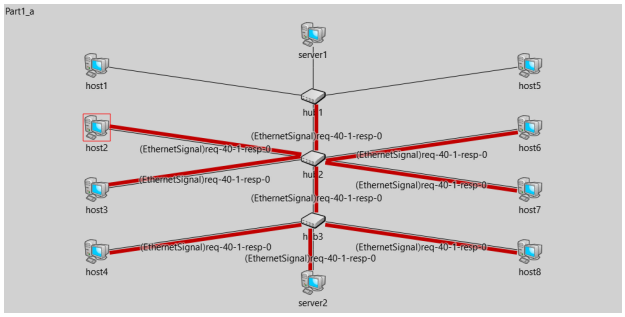
e)

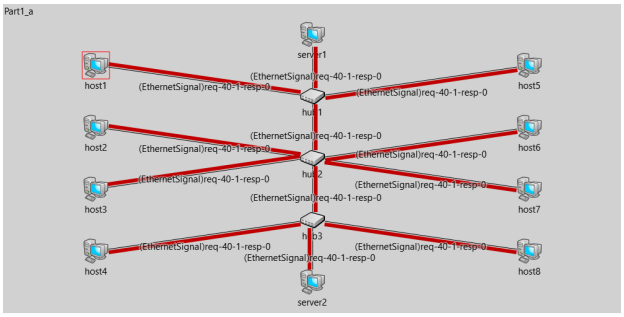
Flow of the frames in the Network using Ethernet Hubs:





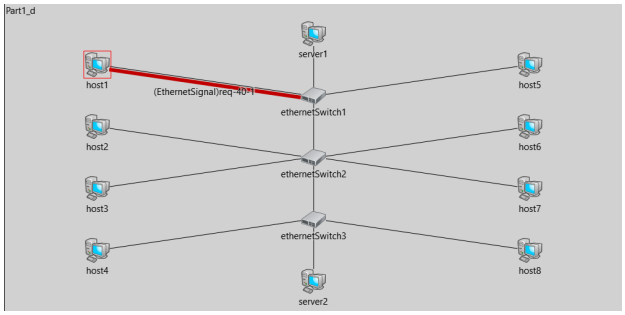


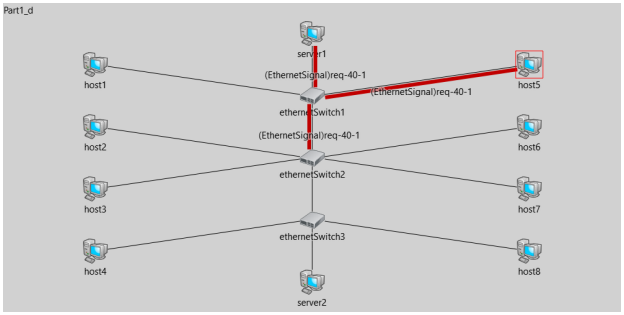


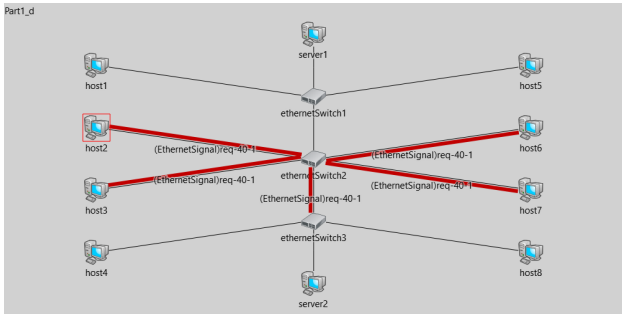


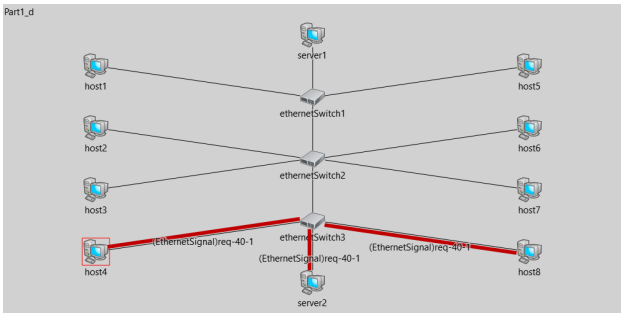
As we can see in the first case, the Hubs send the signal they hear to all other ports of them each single time and they do not learn the host addresses.

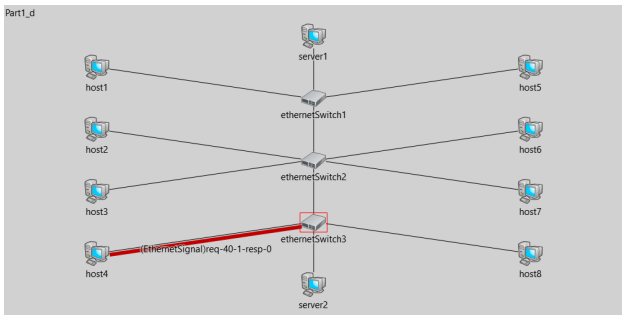
Flow of the frames in the Network using Ethernet Switches

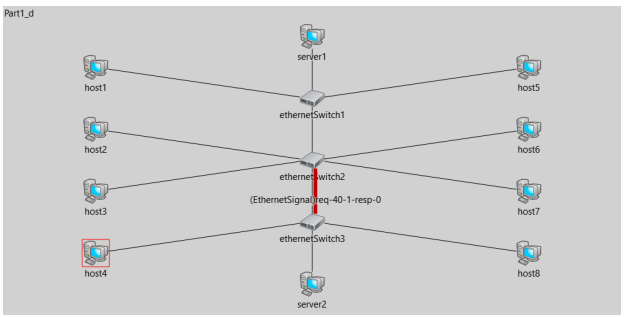


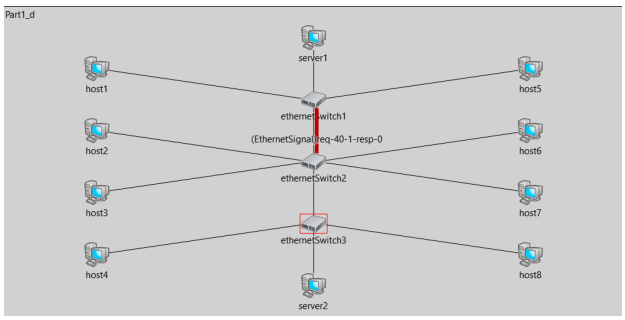




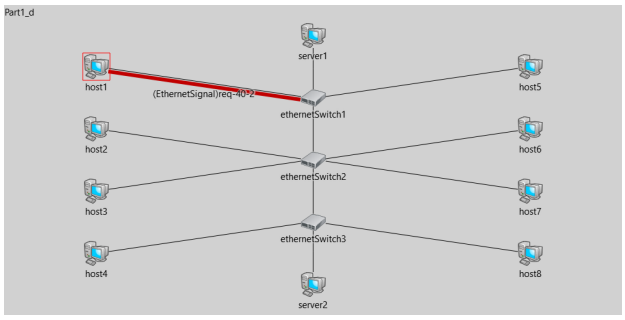


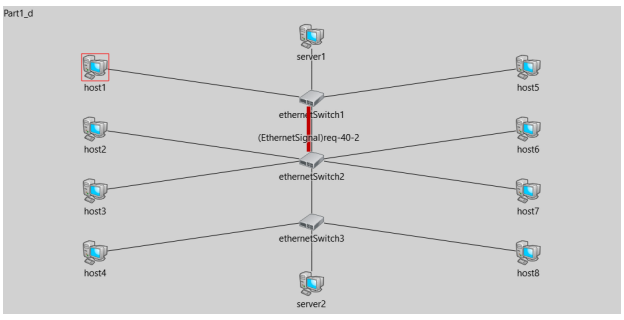


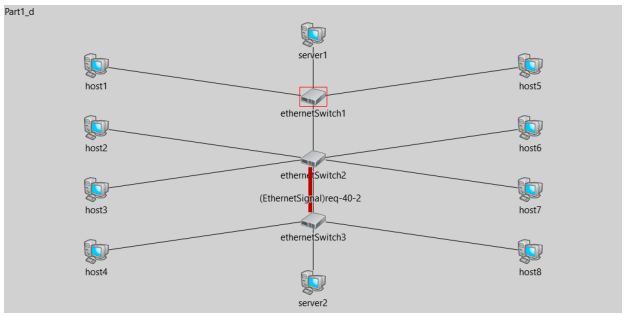


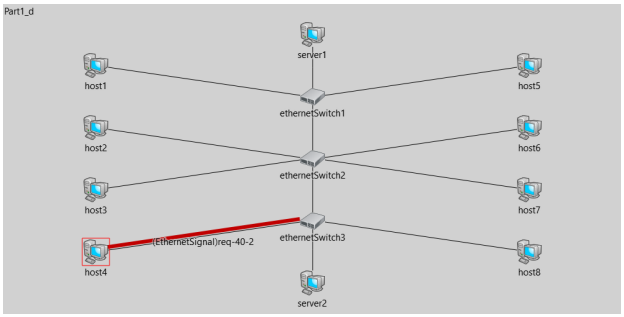


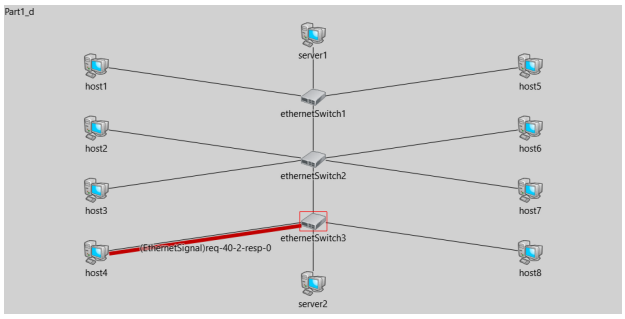


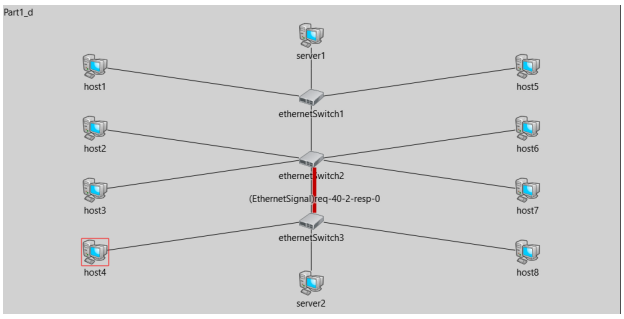


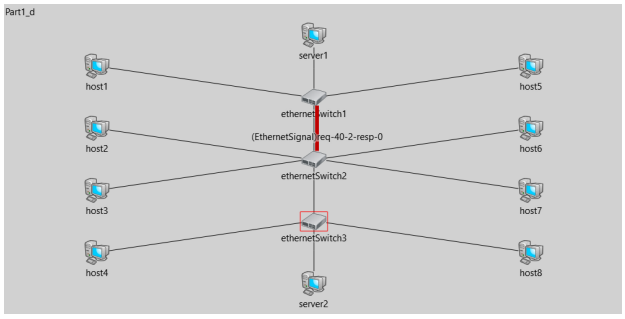


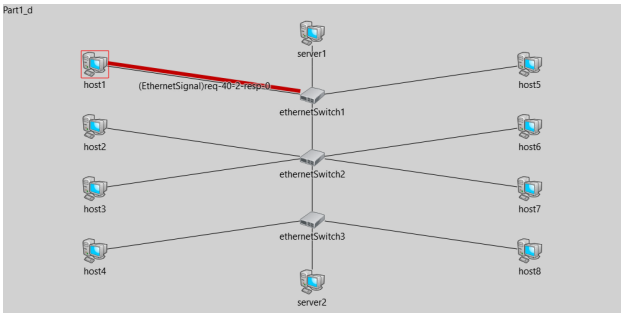












The Switches on the other hand, learn the address of the hosts transmitting frames to them in the first transmission and from that point they do not send the receiving frames to all other ports and they just transmit the frame in the right path.

So we observed the explanations provided in part (a) in simulation.

1.2 CSMA/CD

In this part we ought to investigate CSMA/CD behavior. The topology of the network is shown in Figure 13.

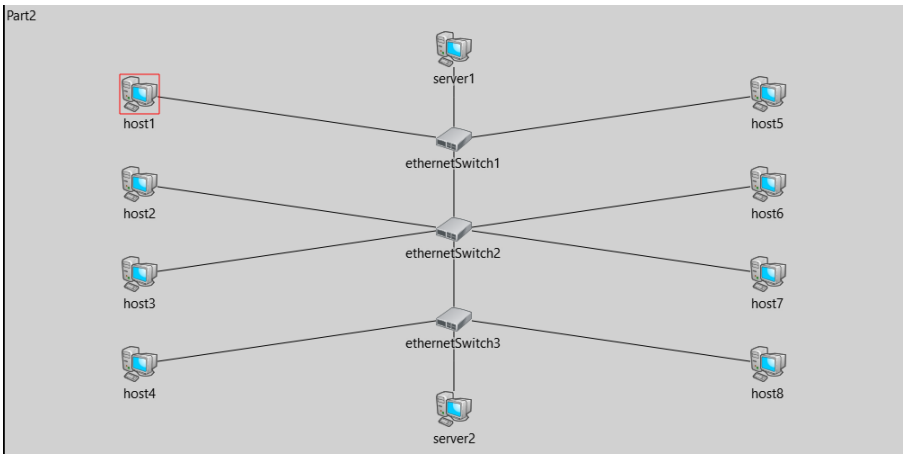


Figure 13: Network description of 1.2

It is demanded that the odd-numbered hosts send requests to *server1* and the even-numbered ones to *server2*. Also I should make sure that the odd-numbered hosts do not support CSMA/CD but other nodes use CSMA/CD.

The code for this simulation configuration is provided in the following figure.

```
49 [Q2]
50 network = topology.Part2
51 sim-time-limit = 10s
52
53 #Part2.*.cli.sendInterval = exponential(50ms)
54 #Part2.*.cli.reqLength = intuniform(800B, 1200B)
55 #Part2.*.cli.respLength = intuniform(10B, 30B)
56
57 Part2.host1.csmacdSupport = false
58 Part2.host3.csmacdSupport = false
59 Part2.host5.csmacdSupport = false
60 Part2.host7.csmacdSupport = false
61
62 Part2.host2.cli.destAddress = "server2"
63 Part2.host4.cli.destAddress = "server2"
64 Part2.host6.cli.destAddress = "server2"
65 Part2.host8.cli.destAddress = "server2"
66 Part2.host1.cli.destAddress = "server1"
67 Part2.host3.cli.destAddress = "server1"
68 Part2.host5.cli.destAddress = "server1"
69 Part2.host7.cli.destAddress = "server1"
```

Figure 14: Initialization code for the needed configurations

I set the simulation time to 10 seconds so that there is enough time for several transmissions from each node.

The results were as below.

Experiment	Measurement	Replication	Module	Name	Value
Q2		#0	Part2.host1.eth.mac	rx channel idle (%)	~ 99.846 3
Q2		#0	Part2.host2.eth.mac	rx channel idle (%)	~ 99.855 8
Q2		#0	Part2.host3.eth.mac	rx channel idle (%)	~ 99.854 7
Q2		#0	Part2.host4.eth.mac	rx channel idle (%)	~ 99.837 9
Q2		#0	Part2.host5.eth.mac	rx channel idle (%)	~ 99.837 9
Q2		#0	Part2.host6.eth.mac	rx channel idle (%)	~ 99.821
Q2		#0	Part2.host7.eth.mac	rx channel idle (%)	~ 99.805 2
Q2		#0	Part2.host8.eth.mac	rx channel idle (%)	~ 99.829 5
Q2		#0	Part2.server1.eth.mac	rx channel idle (%)	~ 99.919 5
Q2		#0	Part2.server2.eth.mac	rx channel idle (%)	~ 99.919 5
Q2		#0	Part2.ethernetSwitch1.eth[0].mac	rx channel idle (%)	~ 99.981 4
Q2		#0	Part2.ethernetSwitch1.eth[1].mac	rx channel idle (%)	~ 99.980 4
Q2		#0	Part2.ethernetSwitch1.eth[2].mac	rx channel idle (%)	~ 99.351 4
Q2		#0	Part2.ethernetSwitch1.eth[3].mac	rx channel idle (%)	~ 99.957 7
Q2		#0	Part2.ethernetSwitch2.eth[0].mac	rx channel idle (%)	~ 99.982 5
Q2		#0	Part2.ethernetSwitch2.eth[1].mac	rx channel idle (%)	~ 99.982 5
Q2		#0	Part2.ethernetSwitch2.eth[2].mac	rx channel idle (%)	~ 99.978 3
Q2		#0	Part2.ethernetSwitch2.eth[3].mac	rx channel idle (%)	~ 99.976 3
Q2		#0	Part2.ethernetSwitch2.eth[4].mac	rx channel idle (%)	~ 99.663
Q2		#0	Part2.ethernetSwitch2.eth[5].mac	rx channel idle (%)	~ 99.679 9
Q2		#0	Part2.ethernetSwitch3.eth[0].mac	rx channel idle (%)	~ 99.980 4
Q2		#0	Part2.ethernetSwitch3.eth[1].mac	rx channel idle (%)	~ 99.979 4
Q2		#0	Part2.ethernetSwitch3.eth[2].mac	rx channel idle (%)	~ 99.351 4
Q2		#0	Part2.ethernetSwitch3.eth[3].mac	rx channel idle (%)	~ 99.959 8

Figure 15: rx channel idle

Experiment	Measurement	Replication	Module	Name	Value
Q2		#0	Part2.host1.eth.mac	rx channel utilization (%)	0.153 696
Q2		#0	Part2.host2.eth.mac	rx channel utilization (%)	0.144 24
Q2		#0	Part2.host3.eth.mac	rx channel utilization (%)	0.145 272
Q2		#0	Part2.host4.eth.mac	rx channel utilization (%)	0.162 12
Q2		#0	Part2.host5.eth.mac	rx channel utilization (%)	0.162 12
Q2		#0	Part2.host6.eth.mac	rx channel utilization (%)	0.178 968
Q2		#0	Part2.host7.eth.mac	rx channel utilization (%)	0.194 784
Q2		#0	Part2.host8.eth.mac	rx channel utilization (%)	0.170 544
Q2		#0	Part2.server1.eth.mac	rx channel utilization (%)	0.080 496
Q2		#0	Part2.server2.eth.mac	rx channel utilization (%)	0.080 496
Q2		#0	Part2.ethernetSwitch1.eth[0].mac	rx channel utilization (%)	0.018 576
Q2		#0	Part2.ethernetSwitch1.eth[1].mac	rx channel utilization (%)	0.019 608
Q2		#0	Part2.ethernetSwitch1.eth[2].mac	rx channel utilization (%)	0.648 648
Q2		#0	Part2.ethernetSwitch1.eth[3].mac	rx channel utilization (%)	0.042 312
Q2		#0	Part2.ethernetSwitch2.eth[0].mac	rx channel utilization (%)	0.017 544
Q2		#0	Part2.ethernetSwitch2.eth[1].mac	rx channel utilization (%)	0.017 544
Q2		#0	Part2.ethernetSwitch2.eth[2].mac	rx channel utilization (%)	0.021 672
Q2		#0	Part2.ethernetSwitch2.eth[3].mac	rx channel utilization (%)	0.023 736
Q2		#0	Part2.ethernetSwitch2.eth[4].mac	rx channel utilization (%)	0.336 96
Q2		#0	Part2.ethernetSwitch2.eth[5].mac	rx channel utilization (%)	0.320 112
Q2		#0	Part2.ethernetSwitch3.eth[0].mac	rx channel utilization (%)	0.019 608
Q2		#0	Part2.ethernetSwitch3.eth[1].mac	rx channel utilization (%)	0.020 64
Q2		#0	Part2.ethernetSwitch3.eth[2].mac	rx channel utilization (%)	0.648 648
Q2		#0	Part2.ethernetSwitch3.eth[3].mac	rx channel utilization (%)	0.040 248

Figure 16: rx channel utilization

Experiment	Measurement	Replication	Module	Name	Value
Q2		#0	Part2.host2.eth.mac	rx channel collision (%)	0
Q2		#0	Part2.host4.eth.mac	rx channel collision (%)	0
Q2		#0	Part2.host6.eth.mac	rx channel collision (%)	0
Q2		#0	Part2.host8.eth.mac	rx channel collision (%)	0
Q2		#0	Part2.server1.eth.mac	rx channel collision (%)	0
Q2		#0	Part2.server2.eth.mac	rx channel collision (%)	0

Figure 17: rx channel collision

As we observed, the *rx channel collision* is not measured in the odd-numbered nodes because they do not support collision detection.

From the results we see that the rx channel of each node is almost always idle(nearly 99% in each node) and therefore the *rx channel utilization* is extremely low in all of the nodes(less than 1%).

Also no collision is detected in the node supporting CSMA/CD because we have used *Ethernet Switched* and they use isolated full-duplex collision domains in each port.

1.3 MAC Address

a)

In this part we need configure the MAC addresses as given and then display the assigned addresses during simulation. I also have to capture and report each switch's MAC address table.

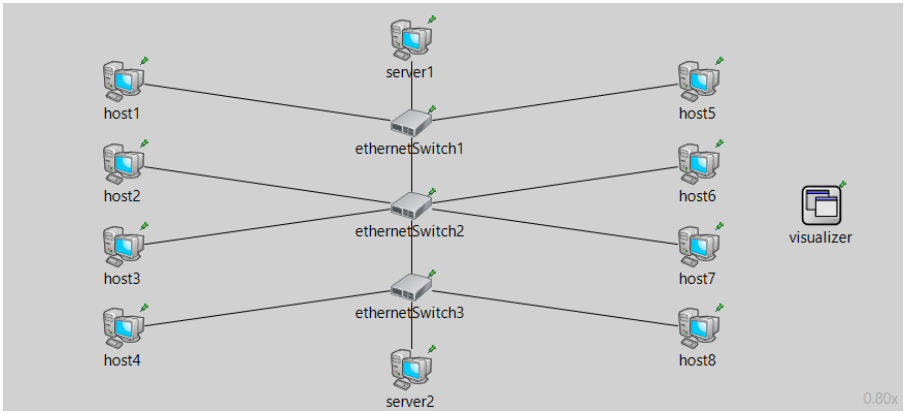


Figure 18: Network description of 1.3

```
71 [Q3]
72 network = topology.Part3
73 sim-time-limit = 1s
74
75 Part3.*.cli.sendInterval = exponential(50ms)
76 Part3.*.cli.reqLength = intuniform(800B, 1200B)
77 Part3.*.cli.respLength = intuniform(10B, 30B)
78
79 Part3.host1.cli.destAddress = "host4"
80 Part3.host3.cli.destAddress = "server2"
81 Part3.host8.cli.destAddress = "server1"
82
83 Part3.visualizer.*.interfaceTableVisualizer.displayInterfaceTables = true
84 Part3.visualizer.*.interfaceTableVisualizer.nodeFilter = "host1 or host2 or host3 or host4 or host5 or host6 or host7 or host8 or server1 or server2"
85 Part3.visualizer.*.interfaceTableVisualizer.format = "%m"
```

Figure 19: Initialization code for the needed configurations

The first step was to set the wanted MAC address to each node. In order to do this I used the code line *@macAddress*. This method didn't work and the MAC addresses were not assigned correctly.

```
network Part3
{
    @display("bgb=1100,500");
    submodules:
        host1: EthernetHost {
            parameters:
            @display("p=150,100");
            @macAddress("00-00-00-00-00-01");
        }
        host2: EthernetHost {
            parameters:
            @display("p=150,200");
            @macAddress("00-00-00-00-00-02");
        }
        host3: EthernetHost {
            parameters:
            @display("p=150,300");
            @macAddress("00-00-00-00-00-03");
        }
```

Figure 20

Then I decided to set the addresses in the initialization file. The added code to the initializations is displayed in Figure 21.

```
86
87 Part3.host1.eth.address = "00-00-00-00-00-01"
88 Part3.host2.eth.address = "00-00-00-00-00-02"
89 Part3.host3.eth.address = "00-00-00-00-00-03"
90 Part3.host4.eth.address = "00-00-00-00-00-04"
91 Part3.host5.eth.address = "00-00-00-00-00-05"
92 Part3.host6.eth.address = "00-00-00-00-00-06"
93 Part3.host7.eth.address = "00-00-00-00-00-07"
94 Part3.host8.eth.address = "00-00-00-00-00-08"
95 Part3.server1.eth.address = "00-00-00-00-00-10"
96 Part3.server2.eth.address = "00-00-00-00-00-20"
```

Figure 21: The code added to the initialization file

The resulting network from simulation is shown in the figure below.

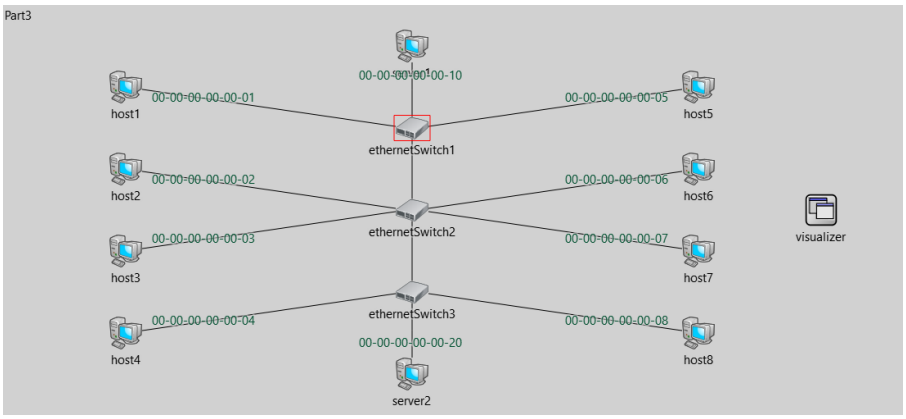


Figure 22

The MAC address of each node is displayed correctly which is what we desired.

b)

After simulating the network, I captured the *forwardingTable* in the *macTable* property of each *EthernetSwitch*. You can see the resulting tables for each Switch in the following figures.

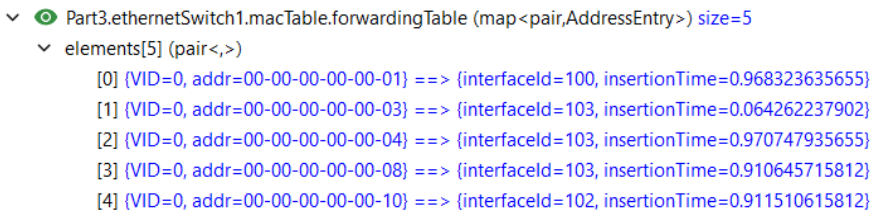


Figure 23: MAC address table of *EthernetSwitch1*

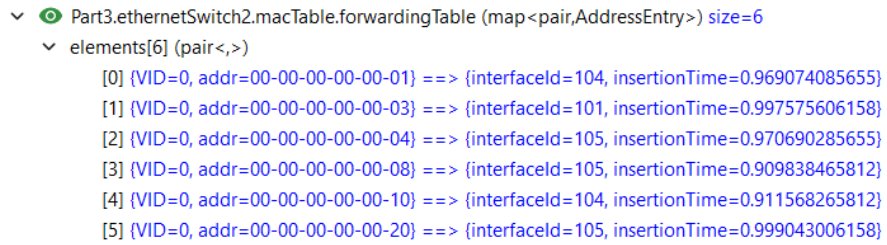


Figure 24: MAC address table of *EthernetSwitch2*

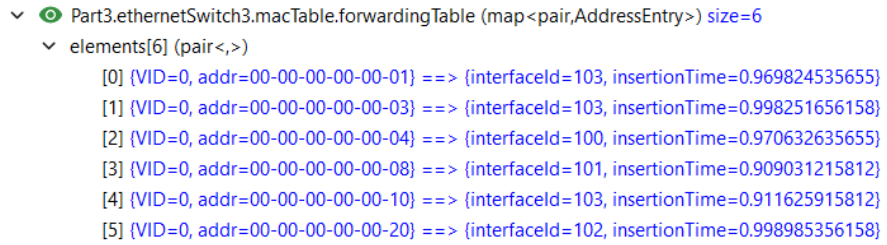


Figure 25: MAC address table of *EthernetSwitch3*

As it is obvious from the results, the addresses are accurately saved in the MAC address tables.

2 VLAN

a)

A Virtual LAN (VLAN) is a logical grouping of devices on a local area network (LAN) that are configured to communicate as if they are on the same physical network, regardless of their actual physical location. VLANs allow network administrators to segment a network into multiple, smaller networks, improving performance, security, and management.

advantages of VLAN:

Improved Security, Better Network Performance, Simplified Management, Flexibility and Scalability, and Reduced Costs

b,c)

The network description for this problem is shown in the following figure.

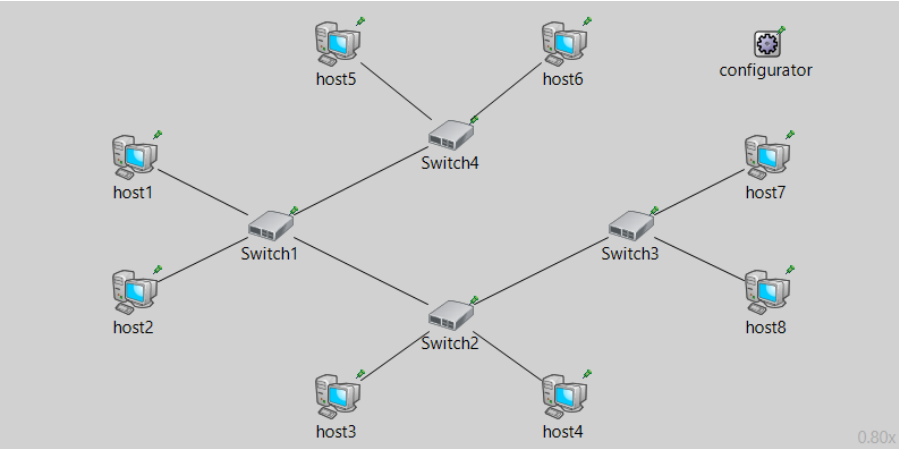


Figure 26: Network description of problem 2

8 *StandardHosts* are transmitting frames and are connected to each other using 4 *EthernetSwitches*. The links connecting the hosts to switches and switches to each other are all 10 meter *Eth10M*s.

Using the VLAN example provided in inet, I assigned the needed vlanIDs to the hosts and also set the accepted in and out vlanIDs for each switch using the codes below in the *.ini* file.

```
[General]
network = topology.VLAN
sim-time-limit = 1.5s
VLAN.host1.numApps = 2
VLAN.host8.numApps = 1
VLAN.host1.app[*].typename = "PingApp"
VLAN.host8.app[*].typename = "PingApp"
VLAN.host1.app[0].destAddr = "host4"
VLAN.host1.app[1].destAddr = "host7"
VLAN.host8.app[0].destAddr = "host6"
VLAN.host1.app[*].printPing = true

VLAN.*.interfaceTable.displayAddresses = true

VLAN.*.bridging.typename = "BridgingLayer"
VLAN.*.bridging.macTableModule = ""
VLAN.switch*.bridging.directionReverser.reverser.forwardPcp = false
VLAN.host*.bridging.directionReverser.typename = ""
VLAN.host*.bridging.interfaceRelay.typename = ""
VLAN.*.ieee8021q.typename = "Ieee8021qProtocol"
```

Figure 27: Base configurations needed for VLANs and PingApp

```
VLAN.Switch*.bridging.vlanPolicy.typename = "VlanPolicyLayer"
VLAN.Switch1.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"eth0" : {"-1" : [10,20,30]} ,
                                                                 "eth1" : {"-1" : [10,20,30]} ,
                                                                 "eth2" : {"-1" : [10,20,30]} ,
                                                                 "eth3" : {"-1" : [10,20,30]}}
VLAN.Switch1.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"eth0" : [10] , "eth1" : [20] , "eth2" : [10,20,30] , "eth3" : [20,30]}
VLAN.Switch1.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"eth0" : [10] , "eth1" : [20] , "eth2" : [10,20,30] , "eth3" : [20,30]}

VLAN.Switch2.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"eth0" : {"-1" : [10,20,30]} ,
                                                                 "eth1" : {"-1" : [10,20,30]} ,
                                                                 "eth2" : {"-1" : [10,20,30]} ,
                                                                 "eth3" : {"-1" : [10,20,30]}}
VLAN.Switch2.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"eth0" : [20] , "eth1" : [10] , "eth2" : [10,20,30] , "eth3" : [10,30]}
VLAN.Switch2.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"eth0" : [20] , "eth1" : [10] , "eth2" : [10,20,30] , "eth3" : [10,30]}

VLAN.Switch3.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"eth0" : {"-1" : [10,20,30]} ,
                                                                 "eth1" : {"-1" : [10,20,30]} ,
                                                                 "eth2" : {"-1" : [10,20,30]} ,
                                                                 "eth3" : {"-1" : [10,20,30]}}
VLAN.Switch3.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"eth0" : [10] , "eth1" : [30] , "eth2" : [10,20,30]}
VLAN.Switch3.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"eth0" : [10] , "eth1" : [30] , "eth2" : [10,20,30]}

VLAN.Switch4.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"eth0" : {"-1" : [10,20,30]} ,
                                                                 "eth1" : {"-1" : [10,20,30]} ,
                                                                 "eth2" : {"-1" : [10,20,30]} ,
                                                                 "eth3" : {"-1" : [10,20,30]}}
VLAN.Switch4.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"eth0" : [20] , "eth1" : [30] , "eth2" : [10,20,30]}
VLAN.Switch4.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"eth0" : [20] , "eth1" : [30] , "eth2" : [10,20,30]}
```

Figure 28: Switch configurations

```
VLAN.host*.bridging.vlanPolicy.typename = "VlanPolicyLayer"
VLAN.host1.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"eth0" : [10]}
VLAN.host1.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"eth0" : [10]}
VLAN.host1.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"eth0" : {"-1" : 10}}

VLAN.host2.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"eth0" : [20]}
VLAN.host2.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"eth0" : [20]}
VLAN.host2.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"eth0" : {"-1" : 20}}

VLAN.host3.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"eth0" : [20]}
VLAN.host3.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"eth0" : [20]}
VLAN.host3.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"eth0" : {"-1" : 20}}

VLAN.host4.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"eth0" : [10]}
VLAN.host4.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"eth0" : [10]}
VLAN.host4.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"eth0" : {"-1" : 10}}

VLAN.host5.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"eth0" : [20]}
VLAN.host5.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"eth0" : [20]}
VLAN.host5.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"eth0" : {"-1" : 20}}

VLAN.host6.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"eth0" : [30]}
VLAN.host6.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"eth0" : [30]}
VLAN.host6.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"eth0" : {"-1" : 30}}

VLAN.host7.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"eth0" : [10]}
VLAN.host7.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"eth0" : [10]}
VLAN.host7.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"eth0" : {"-1" : 10}}

VLAN.host8.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = {"eth0" : [30]}
VLAN.host8.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = {"eth0" : [30]}
VLAN.host8.bridging.vlanPolicy.outboundMapper.mappedVlanIds = {"eth0" : {"-1" : 30}}
```

Figure 29: Host configurations

Using the codes in the preceding figures, I determined the acceptable input and output vlanIDs for each switch.

I learned from the inet vlan example that the code works like this:

$$VLAN.host*.bridging.vlanPolicy.typename = \text{"}VlanPolicyLayer\text{"}$$

This line applies the *VlanPolicyLayer* to all hosts. It ensures that the hosts will also use VLAN filtering and mapping rules.

$$VLAN.host1.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = \{\text{"}eth0\text{"} : [10]\}$$

This line specifies that *host1* will only accept traffic tagged with vlanID 10 on its *eth0* interface. Any traffic tagged with other vlanIDs will be rejected.

$$VLAN.host1.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = \{\text{"}eth0\text{"} : [10]\}$$

This line ensures that *host1* only sends traffic tagged with vlanID 10 on its *eth0* interface. Any traffic with other VLAN tags will be discarded.

$$VLAN.host1.bridging.vlanPolicy.outboundMapper.mappedVlanIds = \{"eth0" : \{"-1" : 10\}\}$$

This line tells *host1* to tag all outgoing traffic from its *eth0* interface with vlanID 10. The -1 here means that all outgoing traffic will be tagged with VLAN 10.

$$VLAN.Switch1.bridging.vlanPolicy.inboundFilter.acceptedVlanIds = \{"eth0" : [10] , "eth1" : [20] , "eth2" : [10,20,30] , "eth3" : [20,30]\}$$

This line specifies which vlanIDs are accepted for incoming traffic on each of *Switch1*'s interfaces.

$$VLAN.Switch1.bridging.vlanPolicy.outboundFilter.acceptedVlanIds = \{"eth0" : [10] , "eth1" : [20] , "eth2" : [10,20,30] , "eth3" : [20,30]\}$$

This line specifies which vlanIDs can be sent out on each of *Switch1*'s interfaces:

On *eth0*, only VLAN 10 traffic is forwarded.

On *eth1*, only VLAN 20 traffic is forwarded.

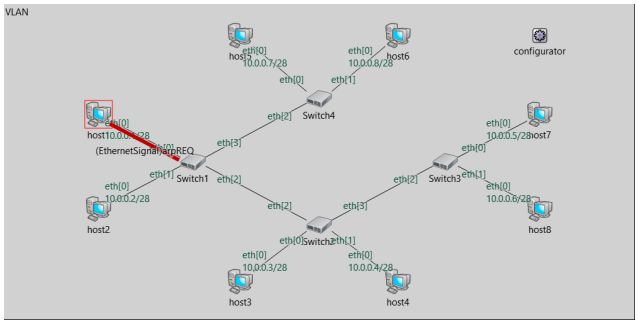
On *eth2*, VLANs 10, 20, and 30 are forwarded.

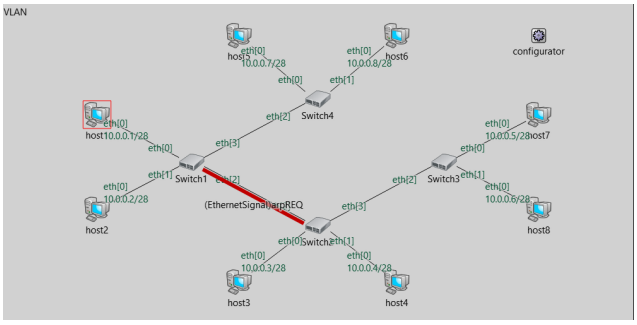
On *eth3*, VLANs 20 and 30 are forwarded.

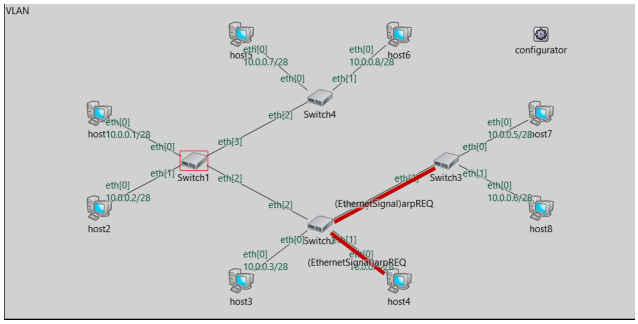
d)

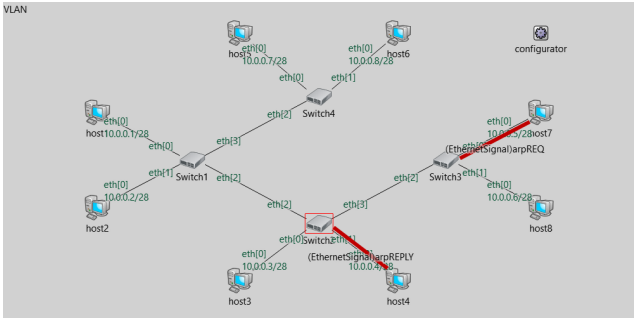
Now is the time to analyze the packet flow in the network and see how the packets are being transmitted.

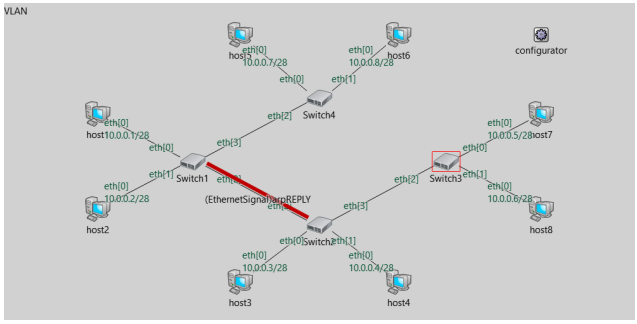
packet flow for *host1* → *host4* communication

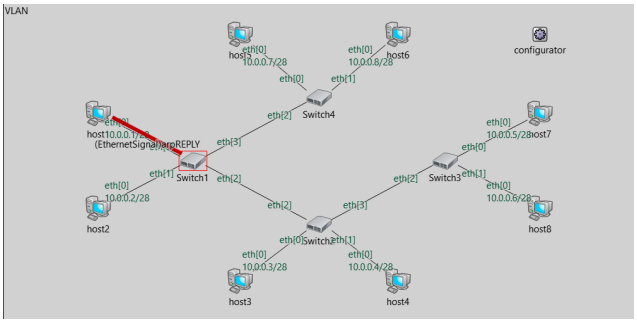


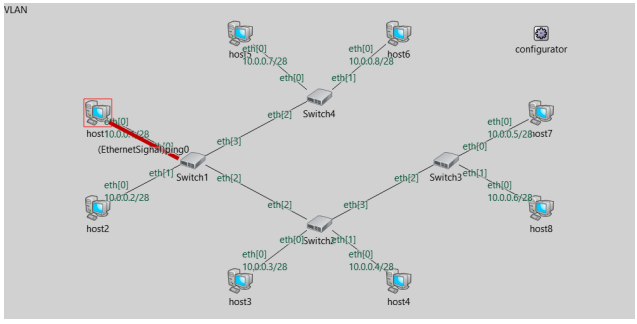


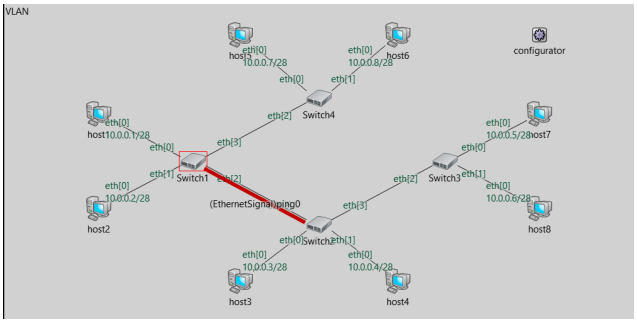


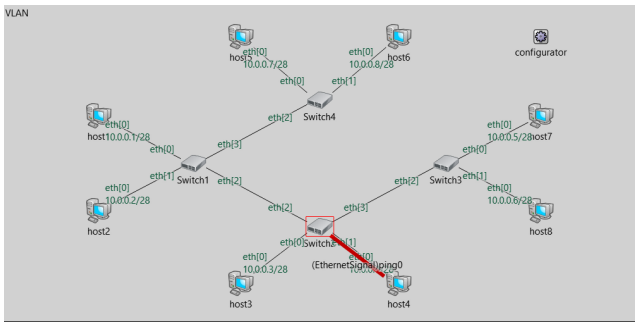


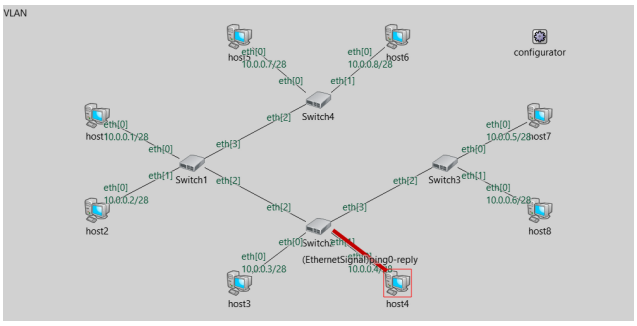

VLAN
configurator
host1
host2
host3
host4
host6
host7
host8
Switch1
Switch2
Switch3
Switch4
(EthernetSignal)arpREPLY

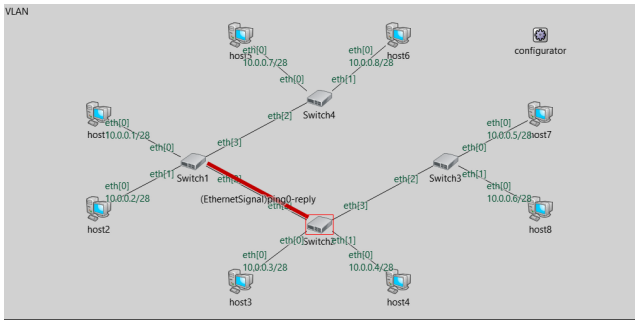

VLAN
configurator
host1
host2
host3
host4
host6
host7
host8
Switch1
Switch2
Switch3
Switch4
(EthernetSignal)arpREPLY

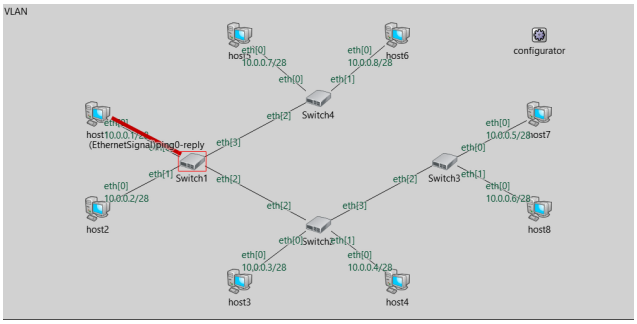

VLAN
configurator
host1
host2
host3
host4
host6
host7
host8
Switch1
Switch2
Switch3
Switch4
(EthernetSignal)ping0


VLAN
configurator
host1
host2
host3
host4
host6
host7
host8
Switch1
Switch2
Switch3
Switch4
(EthernetSignal)ping0

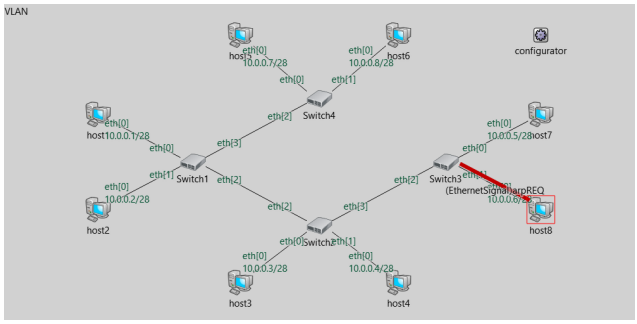

VLAN
configurator
host1
host2
host3
host4
host6
host7
host8
Switch1
Switch2
Switch3
Switch4
(EthernetSignal)ping0

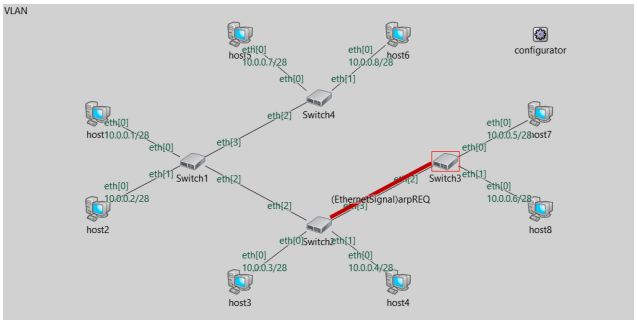

VLAN
configurator
host1
host2
host3
host4
host6
host7
host8
Switch1
Switch2
Switch3
Switch4
(EthernetSignal)ping0-reply

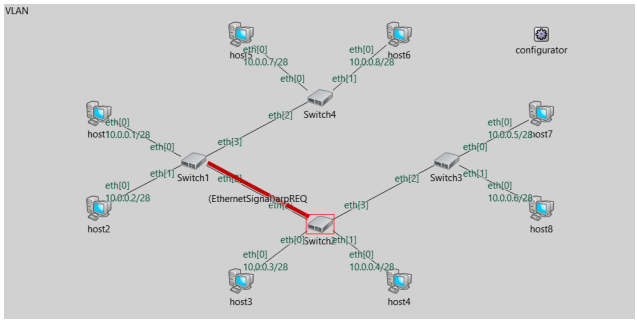

VLAN
configurator
host1
host2
host3
host4
host6
host7
host8
Switch1
Switch2
Switch3
Switch4
(EthernetSignal)ping0-reply

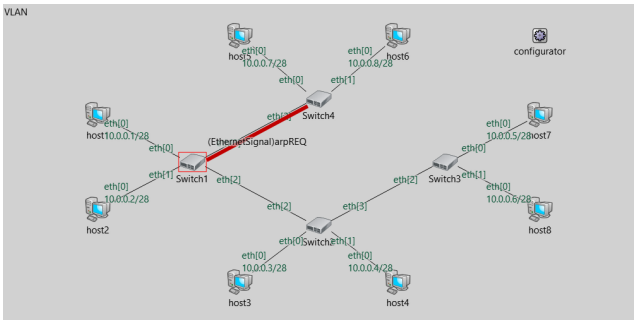

VLAN
configurator
host1
host2
host3
host4
host6
host7
host8
Switch1
Switch2
Switch3
Switch4
(EthernetSignal)ping0-reply

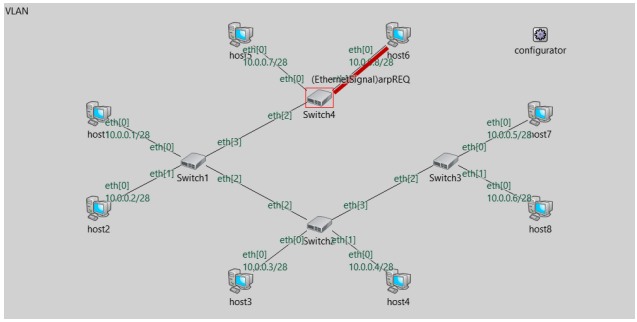
packet flow for *host8* → *host6* communication

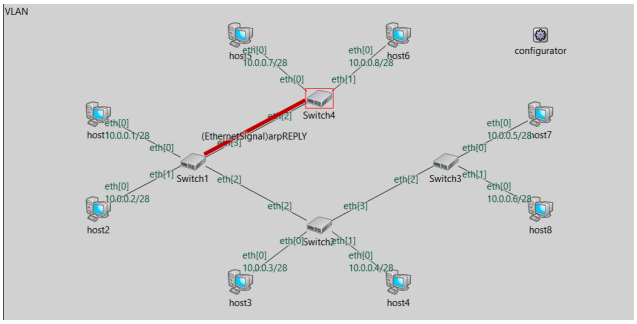


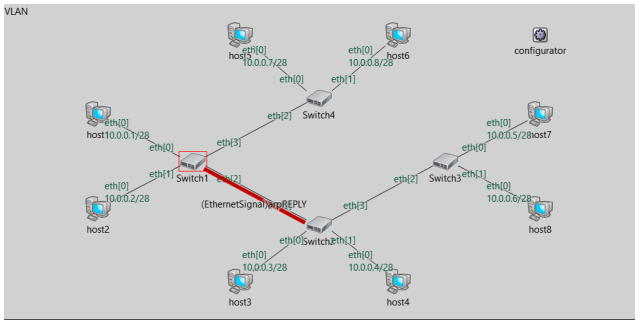


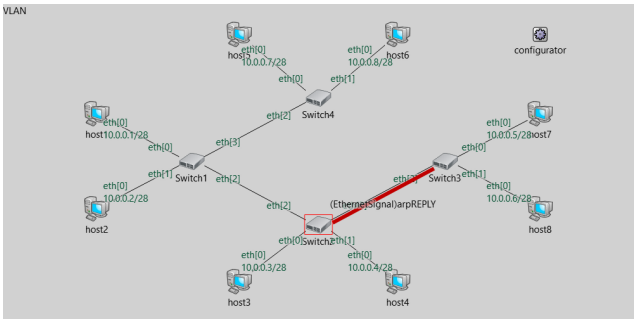


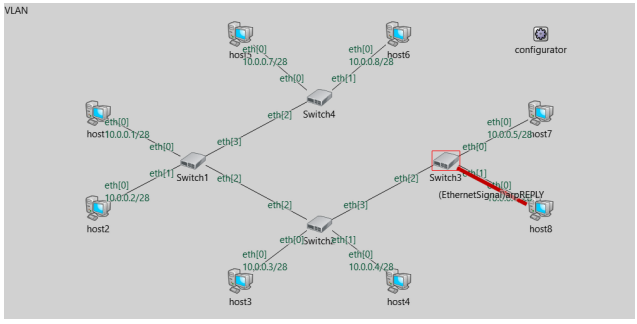


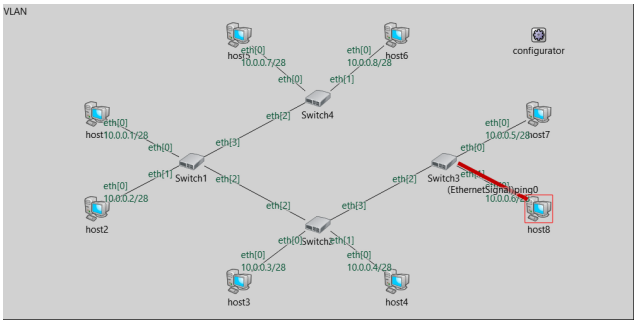


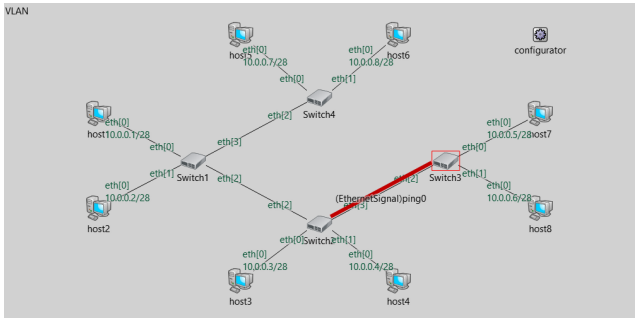


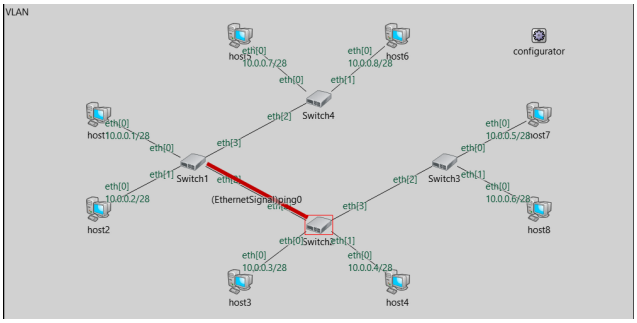


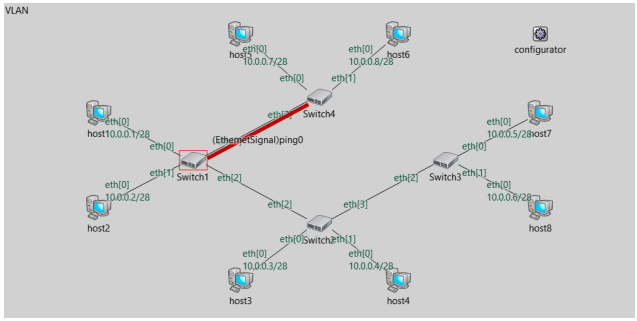


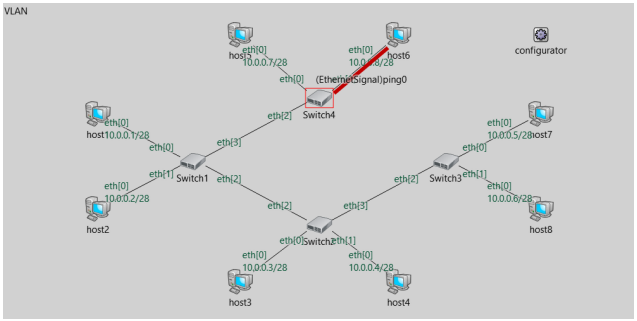


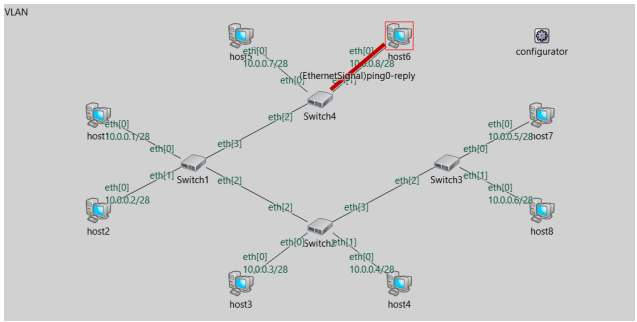


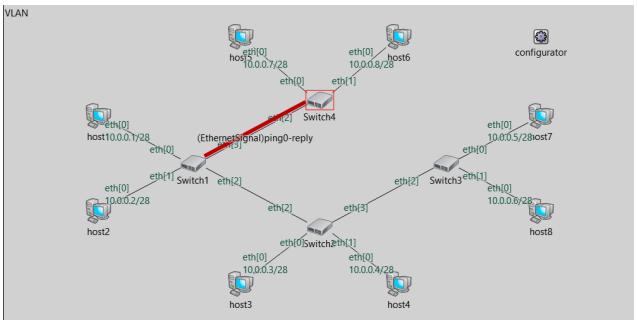


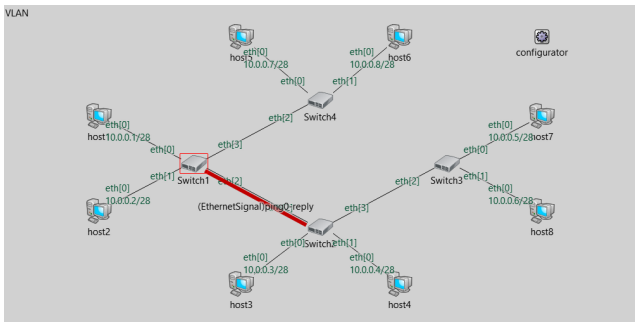


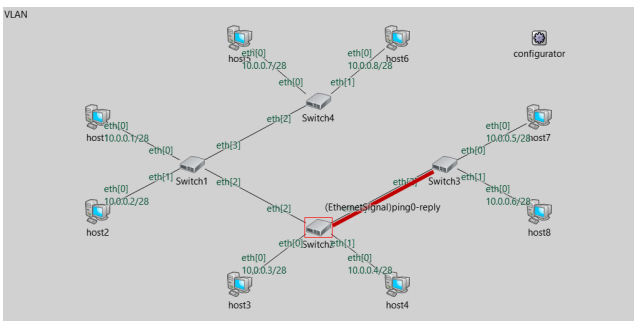












As you can see from the results, the source hosts first send a packet named *arpREQ* to the destination hosts to find the MAC address of the destination host in order to transmit the data.

After the reply from the destination came back to the source host, it starts to send *ping* packets to the known destination.

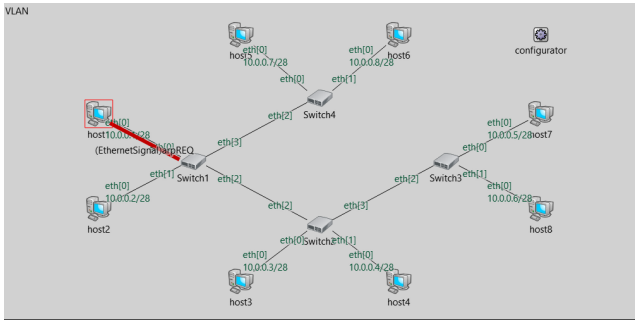
We note that the *arpREQ* packets are sent from every port of each switch than allows transmission of a vlanID similar to the one used in the source host. For example the *arpREQ* packet from *host1* to *host4* is sent from *Switch1* only to *Switch2* because *Switch4* only allows VLANs 20 and 30.

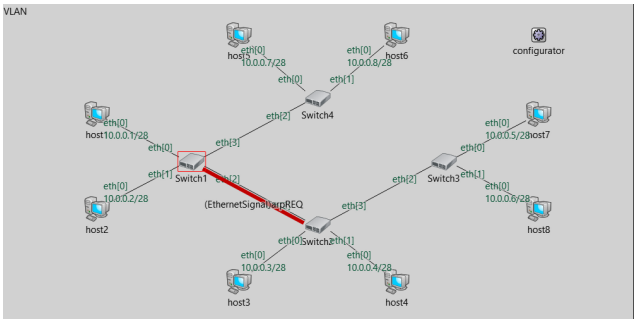
The other thing worth of noting is that after learning the MAC address of the destination and source hosts, the switches only send packets in the correct paths to source and destination and do not send the packets to all nodes accepting packets having valid vlanIDs.

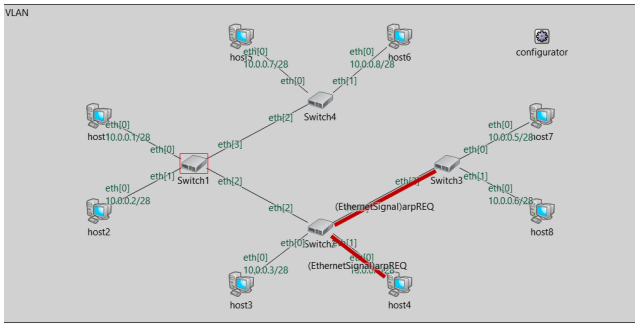
e)

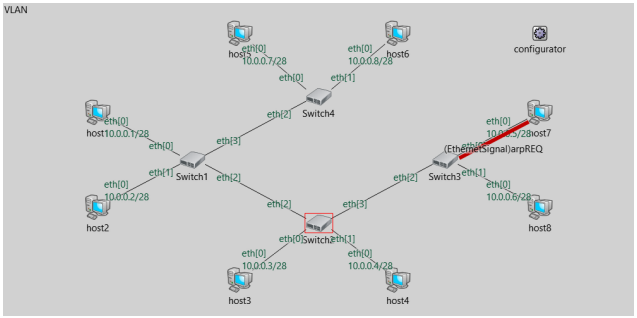
In the following figures I have captured the flow of the traffic moved between *host1* and *host7*.

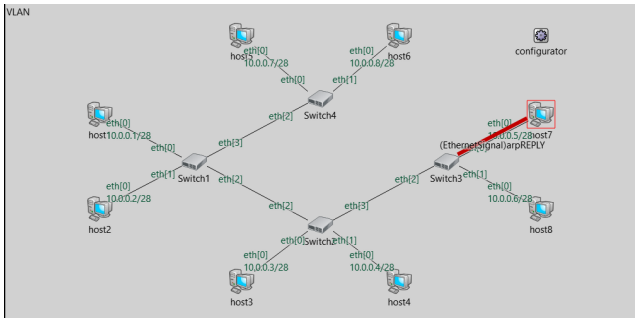
arpREQ packet transmission

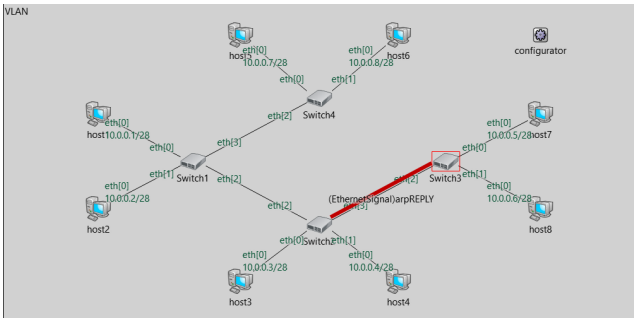


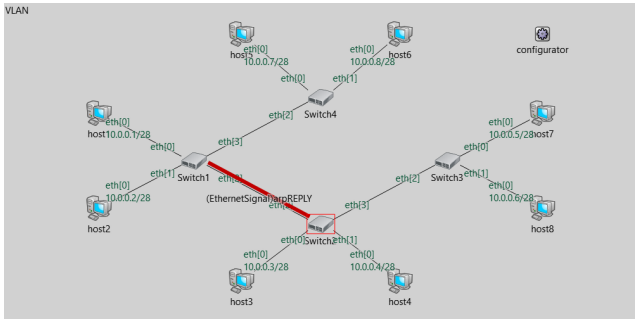


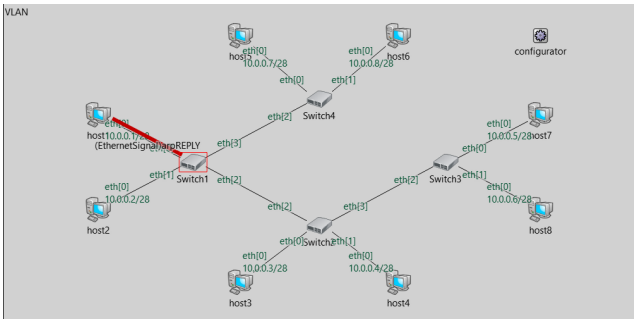










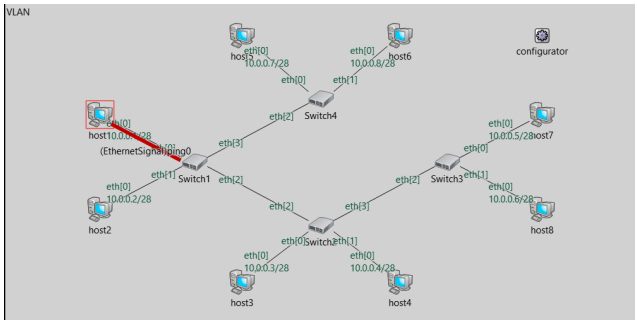


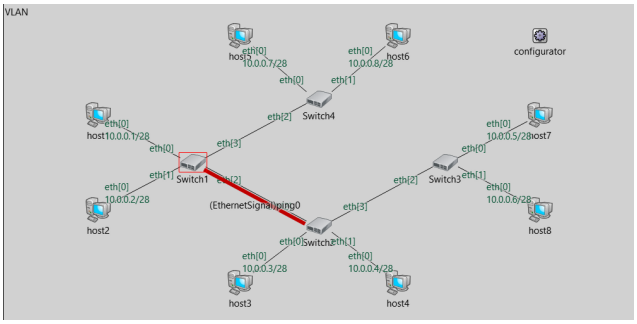
nodes and switches receiving *arpREQ* and *arpREPLY* packets:

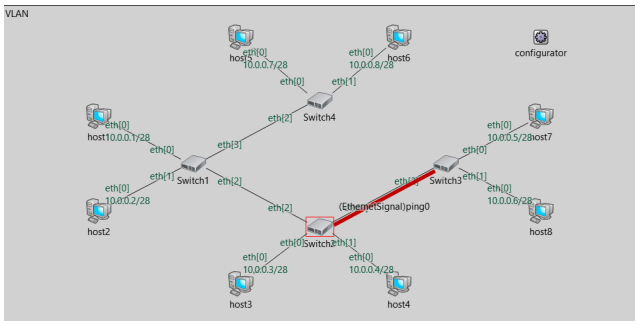
host1 , *Switch1* , *Switch2* , *host4* , *Switch3* , *host7*

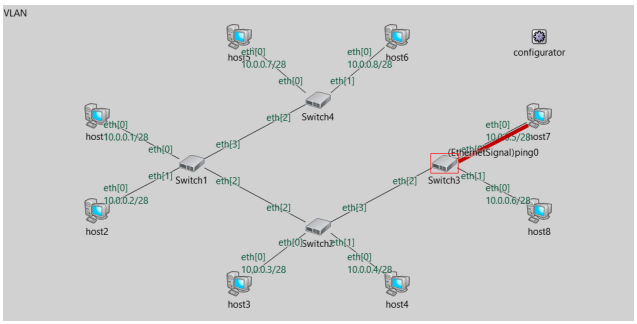
note that *host1* and *host4* receive only *arpREPLY* and *arpREQ* respectively.

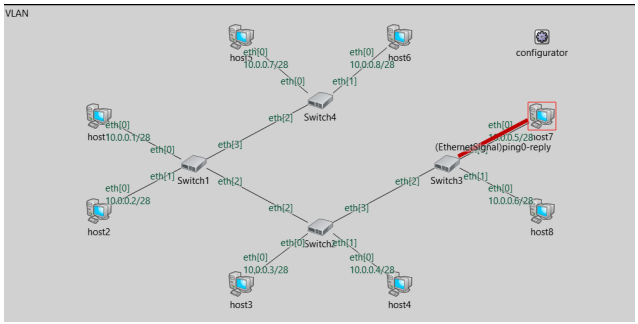
ping packet transmission

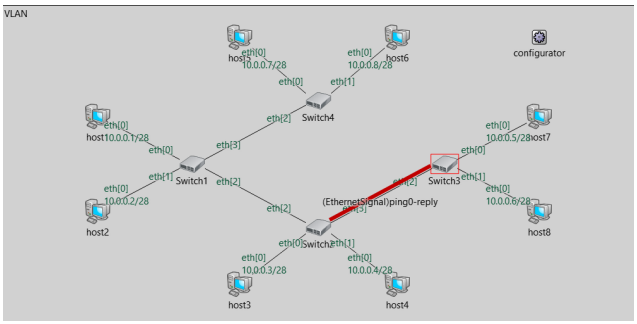


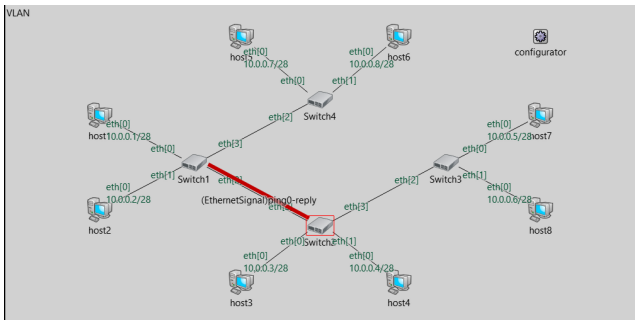


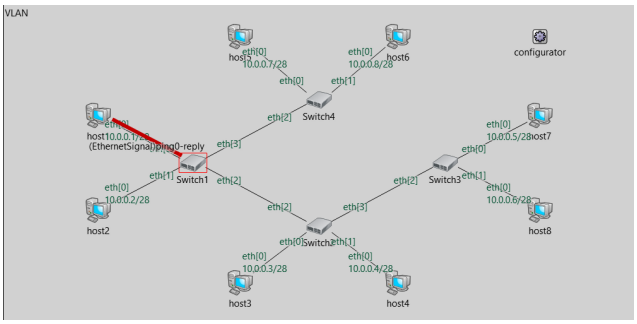












nodes and switches receiving *ping* packets:

Switch1 , *Switch2* , *Switch3* , *host7*

ARP (Address Resolution Protocol) is used to map an IP address to a MAC address in a local network. When a host wants to send data to another host on the same local network, it needs to know the MAC address of the destination host. While the IP address uniquely identifies a host on the network, only the MAC address can be used for actual data transfer on the local link.

When a host wants to communicate with another host on the same network, it checks its ARP cache (a table where the device keeps a record of known IP-to-MAC address mappings).

If the IP address of the destination host is not found in the ARP cache, an *arpREQ* is sent out.

This request is a broadcast message sent to all host on the local network (since the sender doesn't know the destination device's MAC address).

The *arpREQ* contains the sender's IP and MAC address and asks:

"Who has this IP address? Please reply with your MAC address."

The device that owns the IP address in the *arpREQ* responds with an *arpREPLY* which includes its MAC address.

The sender then stores this mapping (IP-to-MAC address) in its ARP cache so future communications to this IP can be sent directly to the MAC address without needing another *arpREQ*.

Therefore the *arpREQ* packets precede *ping* packets because the source needs to find out what the MAC address of the destination host is in order to sent data and communicate.